

Sri Lanka Institute of Information Technology



IT3091: Machine Learning
Year 3, Semester 1- 2026

Practical Answer Submission

Lab Sheet No: 02

Student ID	IT24103352
Student Name	AHAMED M.A.W
Campus/ Center name	SLIIT Kandy UNI
Specialization	Artificial Intelligence

Part A: Dataset Understanding

1. Load the California Housing Dataset and Display

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/ML y3s1/Lab02/CaliforniaHousing.csv')

print("First 5 Records:\n", df.head())
print("\nLast 5 Records:\n", df.tail())
print("\nDataset Shape:", df.shape)
print("\nColumn Names:", df.columns.tolist())
```

First 5 Records:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseValue
0	5.3668	27.0	6.702899	1.028986	1793.0	3.248188	33.92	-117.98	2.198
1	3.4919	44.0	5.138418	1.031073	1137.0	3.211864	37.75	-122.50	2.718
2	1.6196	38.0	3.830116	1.077220	732.0	2.826255	37.75	-122.17	0.851
3	4.0926	42.0	4.525114	0.981735	717.0	3.273973	33.89	-118.09	1.644
4	3.9063	41.0	4.633540	0.900621	387.0	2.403727	37.69	-122.10	1.784

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseValue
0	5.3668	27.0	6.702899	1.028986	1793.0	3.248188	33.92	-117.98	2.198
1	3.4919	44.0	5.138418	1.031073	1137.0	3.211864	37.75	-122.50	2.718
2	1.6196	38.0	3.830116	1.077220	732.0	2.826255	37.75	-122.17	0.851
3	4.0926	42.0	4.525114	0.981735	717.0	3.273973	33.89	-118.09	1.644
4	3.9063	41.0	4.633540	0.900621	387.0	2.403727	37.69	-122.10	1.784

Last 5 Records:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseValue
95	4.1050	16.0	6.035963	1.041763	2515.0	2.917633	32.83	-116.91	1.744
96	6.6073	17.0	6.771499	0.939803	2734.0	3.358722	34.29	-118.75	2.581
97	5.0758	35.0	6.378000	1.054000	1727.0	3.454000	34.05	-117.91	2.111
98	5.5061	17.0	6.923129	1.134862	4956.0	3.341875	34.46	-118.50	2.394
99	2.5313	30.0	5.039384	1.193493	NaN	2.679795	35.14	NaN	0.458

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseValue
95	4.1050	16.0	6.035963	1.041763	2515.0	2.917633	32.83	-116.91	1.744
96	6.6073	17.0	6.771499	0.939803	2734.0	3.358722	34.29	-118.75	2.581
97	5.0758	35.0	6.378000	1.054000	1727.0	3.454000	34.05	-117.91	2.111
98	5.5061	17.0	6.923129	1.134862	4956.0	3.341875	34.46	-118.50	2.394
99	2.5313	30.0	5.039384	1.193493	NaN	2.679795	35.14	NaN	0.458

Dataset Shape: (100, 9)

Column Names: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude', 'MedHouseValue']

A.2 - Number of samples, features, dataset type

```
print("Number of samples:", df.shape[0])
print("Number of features:", df.shape[1] - 1) # target eka ain karala
print("Dataset type: Regression dataset (target variable continuous)")
```

Number of samples: 96

Number of features: 8

Dataset type: Regression dataset (target variable continuous)

```
# A.3 - Identify the data set information by using df.info()
df.info()
```

```
print("Numerical columns:", df.select_dtypes(include='number').columns.tolist())
print("Categorical columns:", df.select_dtypes(include='object').columns.tolist())
print("Target variable: MedHouseValue")
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 96 entries, 0 to 99
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   MedInc              96 non-null     float64
 1   HouseAge            96 non-null     float64
 2   AveRooms            96 non-null     float64
 3   AveBedrms           96 non-null     float64
 4   Population          96 non-null     float64
 5   AveOccup            96 non-null     float64
 6   Latitude            96 non-null     float64
 7   Longitude           96 non-null     float64
 8   MedHouseValue       96 non-null     float64
dtypes: float64(9)
memory usage: 7.5 KB
Numerical columns: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude', 'MedHouseValue']
Categorical columns: []
Target variable: MedHouseValue
```

```
# A.4 Display the statistical summary
```

```
summary = df.describe().T
summary['range'] = summary['max'] - summary['min']
print(summary)

print("\nHighest mean feature:", summary['mean'].idxmax())
print("Largest range feature:", summary['range'].idxmax())
print("Largest std feature:", summary['std'].idxmax())
```

	count	mean	std	min	25%	\
MedInc	96.0	3.609200	1.439182	1.225400	2.590300	
HouseAge	96.0	28.947917	11.673064	4.000000	18.000000	
AveRooms	95.0	5.073789	1.159517	2.096692	4.171829	
AveBedrms	96.0	1.049349	0.119091	0.877301	1.006378	
Population	93.0	1500.881720	1012.192636	240.000000	785.000000	
AveOccup	95.0	3.031639	0.650918	1.360332	2.622609	
Latitude	95.0	35.591263	2.196050	32.710000	33.930000	
Longitude	92.0	-119.549239	2.031340	-124.130000	-121.585000	
MedHouseValue	96.0	1.948698	1.085607	0.458000	1.290750	

	50%	75%	max	range
MedInc	3.474950	4.263950	8.113200	6.887800
HouseAge	29.000000	37.000000	52.000000	48.000000
AveRooms	5.152500	5.863680	8.970968	6.874276
AveBedrms	1.030534	1.077766	1.990323	1.113022
Population	1263.000000	1959.000000	5587.000000	5347.000000
AveOccup	2.922500	3.350298	4.856655	3.496323
Latitude	34.200000	37.550000	40.920000	8.210000
Longitude	-118.545000	-118.010000	-116.910000	7.220000
MedHouseValue	1.623000	2.524000	5.000010	4.542010

```
Highest mean feature: Population
Largest range feature: Population
Largest std feature: Population
```

Part B: Exploratory Data Analysis

1. Check whether the dataset contains any missing values and duplicate records. (If duplicate records exist, remove them and display the updated dataset shape.)

```
## Missing Values & Duplicate Records Check
```

```
# 1. Check for missing values
```

```
print("--- Missing Values ---")
```

```
print(df.isnull().sum())
```

```
# 2. Check for duplicate records
```

```
duplicates = df.duplicated().sum()
```

```
print("\nDuplicate Records Count:", duplicates)
```

```
# 3. If duplicates exist, remove them
```

```
if duplicates > 0:
```

```
    df = df.drop_duplicates()
```

```
    print("Updated Dataset Shape:", df.shape)
```

```
else:
```

```
    print("No duplicates found. Dataset Shape remains:", df.shape)
```

```
--- Missing Values ---
```

```
MedInc      0
```

```
HouseAge    0
```

```
AveRooms    1
```

```
AveBedrms   0
```

```
Population  3
```

```
AveOccup    1
```

```
Latitude    1
```

```
Longitude    4
```

```
MedHouseValue 0
```

```
dtype: int64
```

```
Duplicate Records Count: 0
```

```
No duplicates found. Dataset Shape remains: (96, 9)
```

5

```

# Missing Value Imputation
# Nearly-symmetric features -> mean, Skewed features -> median
df['AveRooms'] = df['AveRooms'].fillna(df['AveRooms'].mean())
df['Latitude'] = df['Latitude'].fillna(df['Latitude'].mean())

df['Population'] = df['Population'].fillna(df['Population'].median())
df['AveOccup'] = df['AveOccup'].fillna(df['AveOccup'].median())
df['Longitude'] = df['Longitude'].fillna(df['Longitude'].median())

print("Missing values after imputation:\n", df.isnull().sum())

```

```

Missing values after imputation:
MedInc          0
HouseAge        0
AveRooms        0
AveBedrms       0
Population      0
AveOccup        0
Latitude        0
Longitude       0
MedHouseValue   0
dtype: int64

```

2. Generate Histograms for the variables and identify,

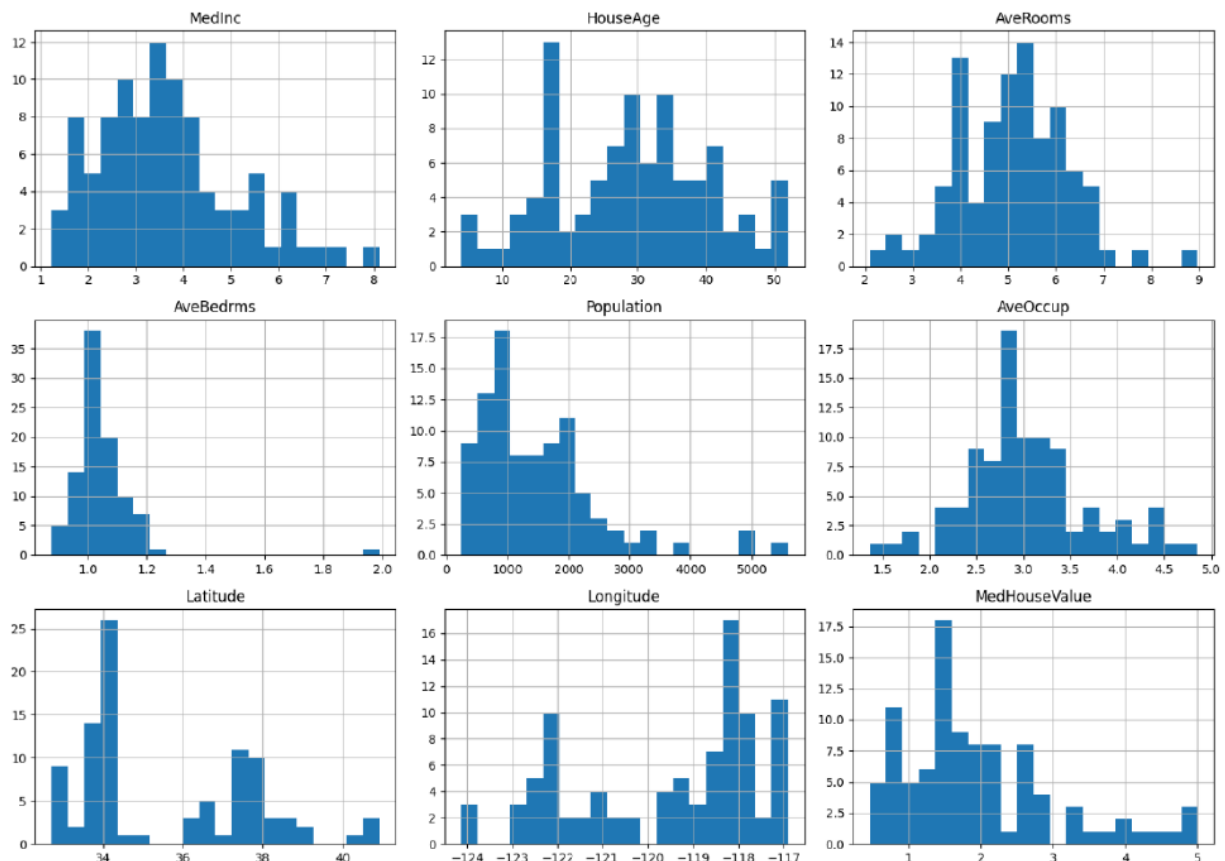
- Which feature looks normally distributed?
- Which feature appears skewed?
- Which feature contains the largest spread?

```
## Histograms Generation & Distribution Analysis
```

```

import matplotlib.pyplot as plt
df.hist(figsize=(14,10), bins=20)
plt.tight_layout()
plt.show()

```



Normally distributed: HouseAge and AveRooms look the closest to a normal distribution .

Skewed features: AveBedrms is very strongly right-skewed, and Population and MedHouseValue are also right-skewed (long tail toward high values).

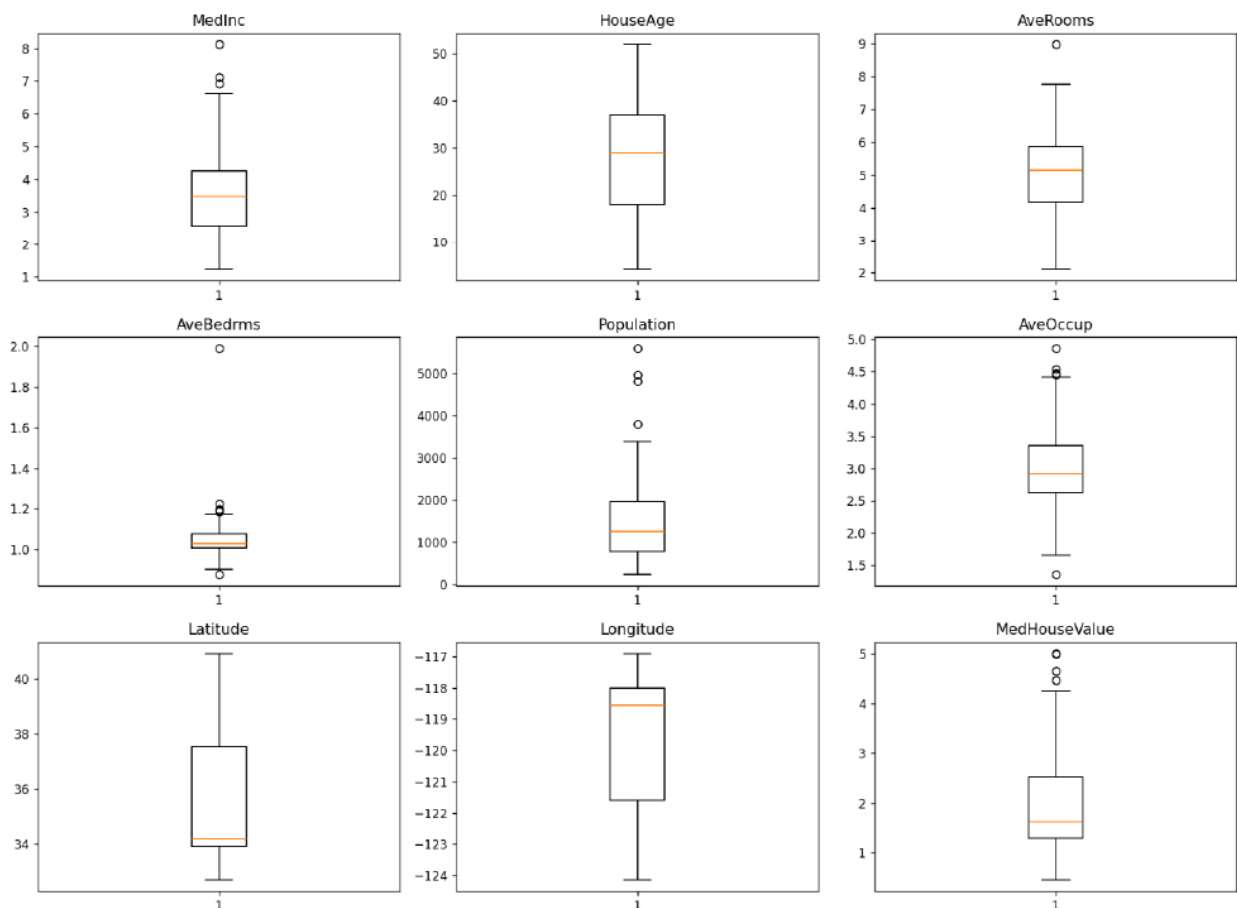
Largest spread: Population has the widest spread of values, consistent with it having the largest standard deviation and range.

3. Choose an appropriate missing value imputation method for each variable based on its distribution and justify your choice.

Missing values were imputed based on each variable's distribution shape: features that are **nearly symmetric** (AveRooms, Latitude) were filled with the **mean**, since it isn't distorted by outliers in symmetric data. Features that are **skewed** (Population, AveOccup, Longitude) were filled with the **median**, since it is more robust to extreme values and better represents the true central tendency of a skewed distribution.

4. Generate separate boxplots for each numerical feature and identify,

- Which features contain outliers?
- Why are outliers important in ML?



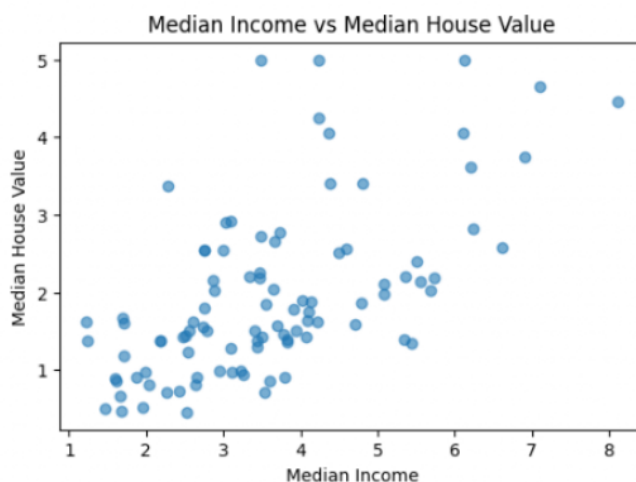
Number of outlier values detected per feature (using the $IQR \times 1.5$ rule):

Feature	Outlier Count
MedInc	2
HouseAge	0
AveRooms	1
AveBedrms	7
Population	5
AveOccup	5
Latitude	0
Longitude	0
MedHouseValue	5

Features with outliers: AveBedrms has the most outliers (7), followed by Population, AveOccup and MedHouseValue (5 each), and MedInc (2) and AveRooms (1). HouseAge, Latitude and Longitude show no extreme outliers under this rule.

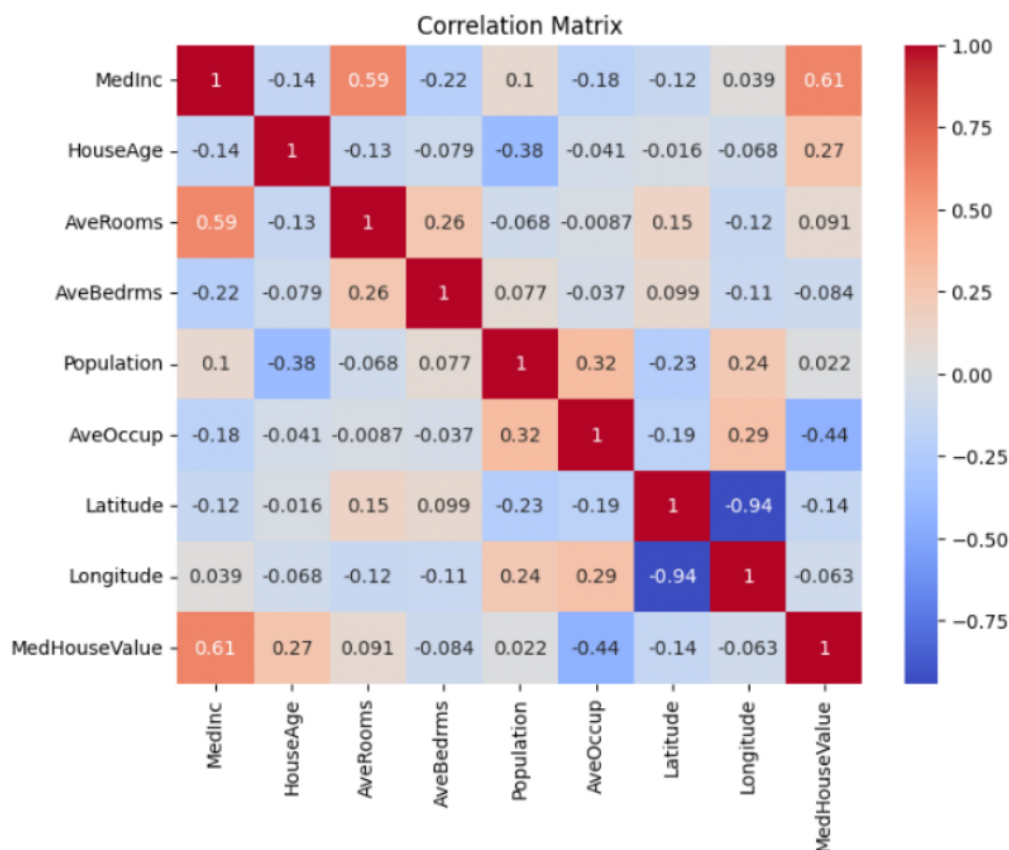
Why outliers matter in ML: Outliers can distort statistics such as the mean and standard deviation, bias model coefficients (especially in linear models that use squared-error loss), and reduce a model's ability to generalize. Detecting them early helps decide whether to cap, transform, remove, or use robust models/scalers to handle them.

```
# 8.5 - Scatter plot: Median Income vs Median House Value
plt.figure(figsize=(6,4))
plt.scatter(df['MedInc'], df['MedHouseValue'], alpha=0.6)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Median Income vs Median House Value')
plt.show()
```



Interpretation: The scatter plot shows a clear positive relationship — as Median Income (MedInc) increases, Median House Value (MedHouseValue) tends to increase as well. This matches the correlation coefficient of about +0.60 between the two variables, making MedInc the strongest single predictor of house value in this dataset. The relationship is roughly linear but with noticeable spread, meaning income alone does not fully determine house value.

```
# B.6 - Correlation heatmap
import seaborn as sns
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



Most positively correlated with the target (MedHouseValue): MedInc (Median Income), with a correlation coefficient of **+0.61**. This confirms that income level is the strongest driver of house value in this dataset.

Least correlated with the target: Population, with a correlation coefficient of only **+0.022** (essentially close to zero). This indicates that population count has almost no linear relationship with median house value.

Strongest correlation between two input variables: Latitude and Longitude, with a correlation coefficient of **-0.94**. This strong negative correlation makes geographical sense, since California's coastline and populated regions run diagonally from the north-west to the south-east — as latitude increases, longitude systematically decreases.

Part C – Train/Test Separation

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

# 1. Split the dataset into Features (X) and Target (y)
X = df.drop(columns=['MedHouseValue'])
y = df['MedHouseValue']

# 2. Perform Train/Test Split (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Part C (1) Outputs ---
print("===== C1: Sample Counts =====")
print(f"Total dataset samples: {len(df)}")
print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples: {X_test.shape[0]}")

# --- Part C (2) Outputs ---
print("\n===== C2: Summary Statistics =====")
stats_df = pd.DataFrame({
    'Train Mean': X_train.mean(),
    'Test Mean': X_test.mean(),
    'Train Std': X_train.std(),
    'Test Std': X_test.std()
})
print(stats_df)

print("\n--- Target Variable (MedHouseValue) Stats ---")
print("Train Target Mean:", y_train.mean(), "| Std:", y_train.std())
print("Test Target Mean: ", y_test.mean(), "| Std:", y_test.std())

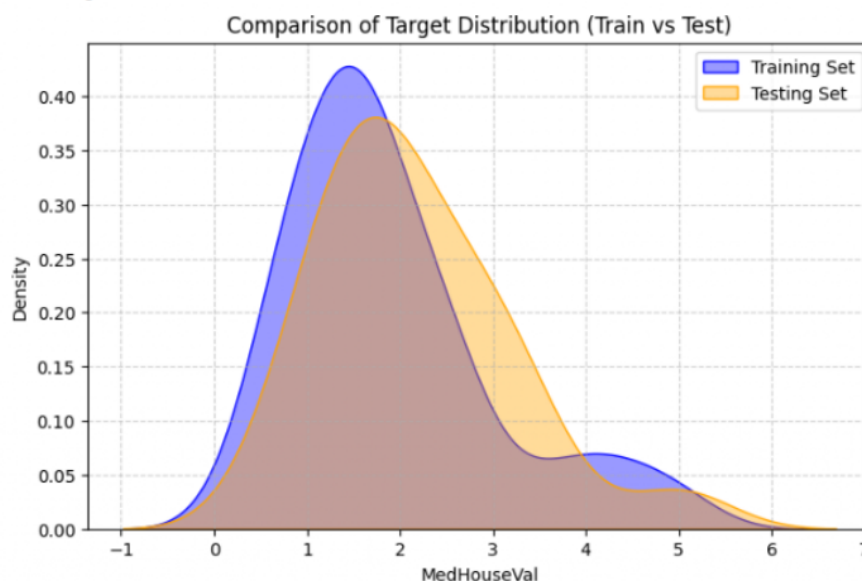
# Visualization - Compare Target Distributions
plt.figure(figsize=(8, 5))
sns.kdeplot(y_train, label='Training Set', fill=True, alpha=0.4, color='blue')
sns.kdeplot(y_test, label='Testing Set', fill=True, alpha=0.4, color='orange')
plt.title('Comparison of Target Distribution (Train vs Test)')
plt.xlabel('MedHouseValue')
plt.ylabel('Density')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
===== C1: Sample Counts =====
Total dataset samples: 96
Training samples: 76
Testing samples: 20
```

```
===== C2: Summary Statistics =====
```

	Train Mean	Test Mean	Train Std	Test Std
MedInc	3.552303	3.825410	1.502549	1.175705
HouseAge	29.184211	28.050000	11.536274	12.445566
AveRooms	5.078142	5.057465	1.166594	1.162226
AveBedrms	1.049236	1.049776	0.127990	0.079058
Population	1496.356164	1517.400000	1010.668183	1043.923692
AveOccup	3.097432	2.784915	0.681084	0.510077
Latitude	35.794933	34.827500	2.202415	2.046859
Longitude	-119.710972	-118.967000	2.039486	1.940008

```
--- Target Variable (MedHouseVal) Stats ---
Train Target Mean: 1.8949739473684208 | Std: 1.101106365728675
Test Target Mean: 2.1528505 | Std: 1.0253607194054863
```



1. Sample Counts

Using `train_test_split()` with `test_size = 0.2` and `random_state = 42` on the cleaned dataset (96 samples after duplicate removal):

- **Training samples:** 76 (80%)
- **Testing samples:** 20 (20%)

2. Comparing Train vs Test Distributions

The summary statistics (mean and standard deviation) for each feature are reasonably close between the training and testing sets. For example:

- **MedInc:** Train mean = 3.55, Test mean = 3.83 — a small difference.
- **HouseAge:** Train mean = 29.18, Test mean = 28.05 — very close.
- **Population:** Train mean = 1496.36, Test mean = 1517.40 — close, with similar standard deviations.
- **Target variable (MedHouseValue):** Train mean = 1.89, Test mean = 2.15, with standard deviations of 1.10 and 1.03 respectively — fairly close.

The KDE (density) plot of the target variable further supports this: both the training and testing distributions follow the same overall shape — a peak around 1–2, a right-skewed tail toward higher values, and a similar range (roughly -1 to 7 after KDE smoothing, though the actual data lies between about 0.46 and 5.0). The two curves overlap substantially, with only minor differences in peak height and small bumps at higher values in the test set.

Conclusion: Yes, the training and testing datasets are **similarly distributed**. Although there are small differences in means and standard deviations across features (expected due to the small sample size of only 96 records), there is no major systematic difference or bias between the two sets. This confirms that the `train_test_split` with `random_state=42` produced a fair, representative split, which is important for ensuring that the model is evaluated on data that resembles what it was trained on.

Part D – Understanding the concepts

1. Why is data understanding essential before training a machine learning model?

Data understanding helps identify what the dataset actually represents — its features, target variable, scale, distribution, and any quality issues such as missing values, duplicates, or outliers. Without this step, a model may be trained on flawed or misleading data, leading to poor performance, incorrect conclusions, or a model that fails to generalize to real-world data. It also guides key decisions such as which preprocessing steps, feature engineering methods, and algorithms are appropriate for the data.

2. What is the difference between univariate and bivariate analysis?

Univariate analysis examines a single variable at a time — for example, a histogram or boxplot of one feature — to understand its distribution, central tendency, and spread.

Bivariate analysis examines the relationship between two variables at a time — for example, a scatter plot of MedInc vs MedHouseValue or a correlation coefficient — to understand how the two variables relate to or influence one another.

3. Why should duplicate records be removed?

Duplicate records give repeated data points extra, unearned weight during model training, which can bias the model's learned parameters toward those repeated patterns. They can also artificially inflate performance metrics if duplicates end up split across both the training and testing sets, giving a false sense of how well the model actually generalizes to new data.

4. Why can missing values reduce model performance?

Many machine learning algorithms cannot process missing values directly and will either throw an error or silently drop those rows, reducing the effective amount of training data available. If missing values are filled incorrectly (e.g., using an inappropriate imputation method), it can introduce bias or noise into the feature, distorting its true distribution and weakening the model's ability to learn accurate patterns.

5. What is data leakage? Give one example.

Data leakage occurs when information from outside the training dataset — particularly information from the test set, or information that would not be available at prediction time — is used to build or tune the model. This leads to unrealistically good performance during evaluation that does not hold up in real-world use.

Example: Scaling or imputing missing values (e.g., using the mean) on the entire dataset *before* splitting it into training and test sets. This causes statistics from the test set to "leak" into the training process, since the training set indirectly gains information about the test set's distribution.

6. Why must train-test splitting be performed before preprocessing?

Splitting the data first ensures that any preprocessing step that learns something from the data — such as computing the mean/median for imputation, or fitting a scaler — is learned only from the training set. The learned parameters (e.g., the training set's mean) are then applied to the test set, rather than being recalculated from it. This keeps the test set as a genuinely unseen sample, giving an honest estimate of how the model will perform on new, real-world data and avoiding data leakage.